
CSE 312

Operating Systems

Spring 2026

Security

© 2013-2026 Yakup Genç

Security and Operating Systems

- What is operating system security?
- How do operating systems contribute to system security?
- Alternatively, if we're trying to develop a secure system, what do we demand of the OS?
- Today's lecture concentrates on what the OS can/should/do

What is Security?

- Informal: Security is keeping unauthorized entities from doing things you don't want them to do.
- More formal: Confidentiality, integrity, availability
- What is the operating system's role?

Threats

Goal	Threat
Data confidentiality	Exposure of data
Data integrity	Tampering with data
System availability	Denial of service
Exclusion of outsiders	System takeover by viruses

Security goals and threats

Intruders

Common categories:

- Casual prying by nontechnical users.
- Snooping by insiders.
- Determined attempts to make money.
- Commercial or military espionage.

Accidental Data Loss

Common causes of accidental data loss:

- Acts of God: fires, floods, earthquakes, wars, riots, or rats gnawing backup tapes.
- Hardware or software errors: CPU malfunctions, unreadable disks or tapes, telecommunication errors, program bugs.
- Human errors: incorrect data entry, wrong tape or CD-ROM mounted, wrong program run, lost disk or tape, or some other mistake.

Internal Roles

- A lot of internal features: privileged mode, memory protection, file access permissions, etc.
- What do these accomplish?
- What is the real goal?

Protecting Whom?

- Internal features protect the operating system against users
- This necessary but not sufficient
- File permissions protect users (and the OS) against other users
- Again, this is necessary but not sufficient

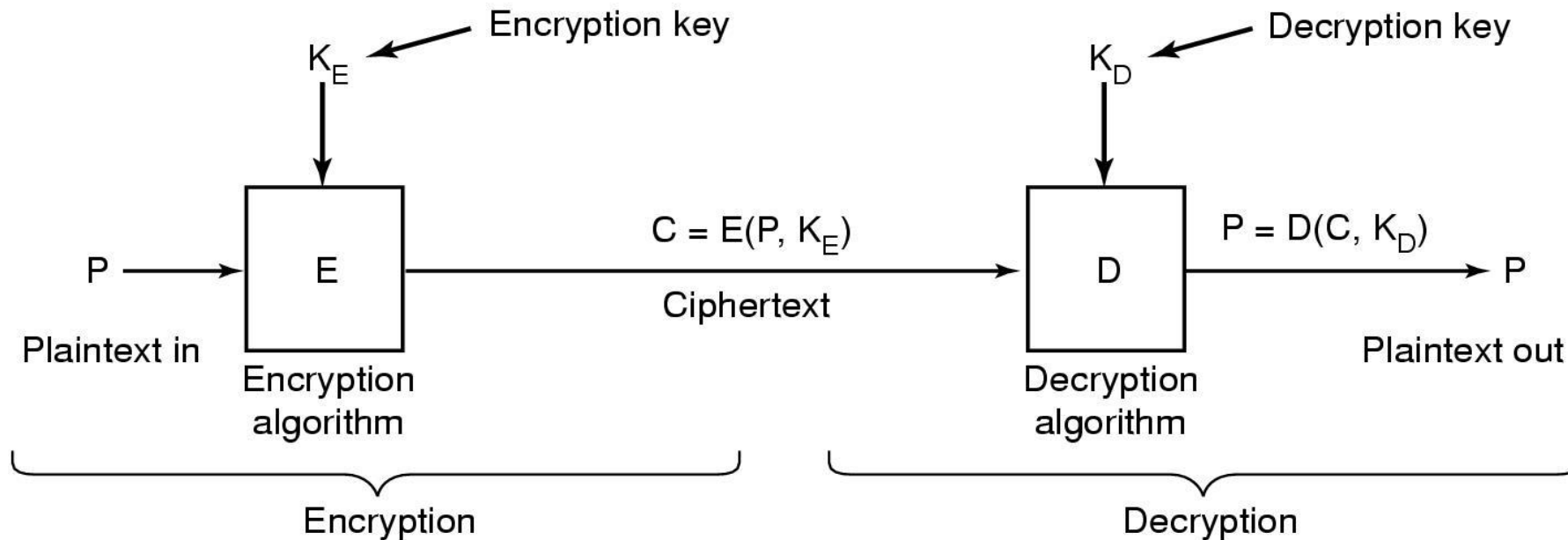
User Authentication

- File permissions are based on user identity, which is based on authentication
- How does an OS authenticate users?
- Many methods: something you know, something you have, something you are

Something You Know: Passwords

- Very common
- Very easily guessed
- Originally stored in plaintext, but that's a very bad idea
- Today, passwords are usually stored hashed
- However — some network authentication schemes, such as challenge/response, require plaintext (or equivalent)

Basics Of Cryptography



Relationship between the plaintext and the ciphertext

Secret-Key Cryptography

Monoalphabetic substitution:

Plaintext: ABCDEFGHIJKLMNOPQRSTUVWXYZ

Ciphertext: QWERTYUIOPASDFGHJKLZXCVBNM

Public-Key Cryptography

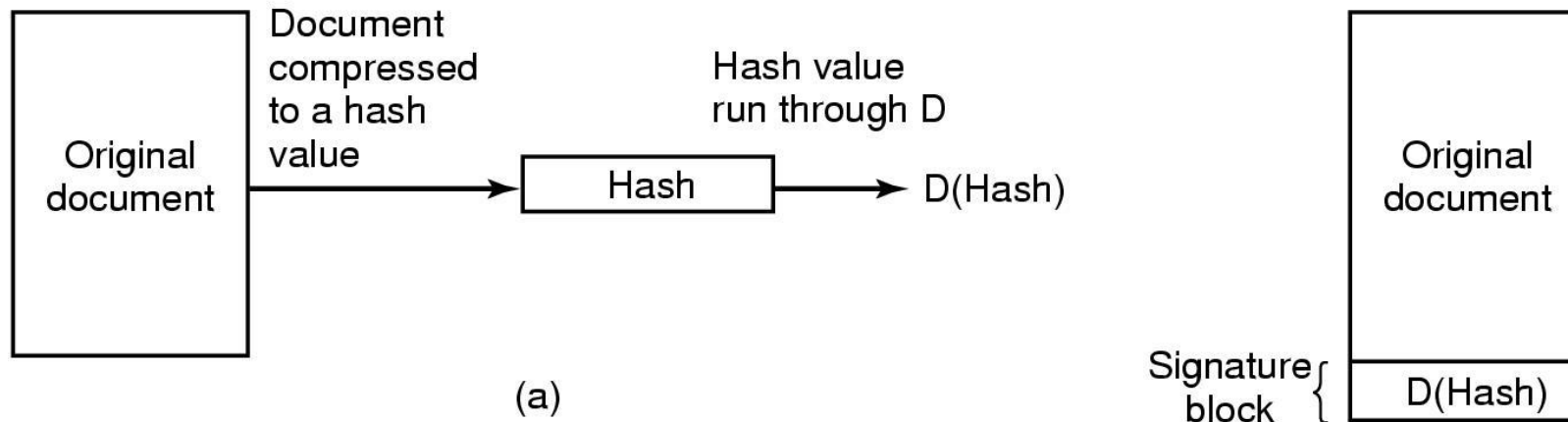
- Encryption makes use of an "easy" operation, such as how much is

$314159265358979 \times 314159265358979?$

- Decryption without the key requires you to perform a hard operation, such as what is the square root of

$3912571506419387090594828508241?$

Digital Signatures



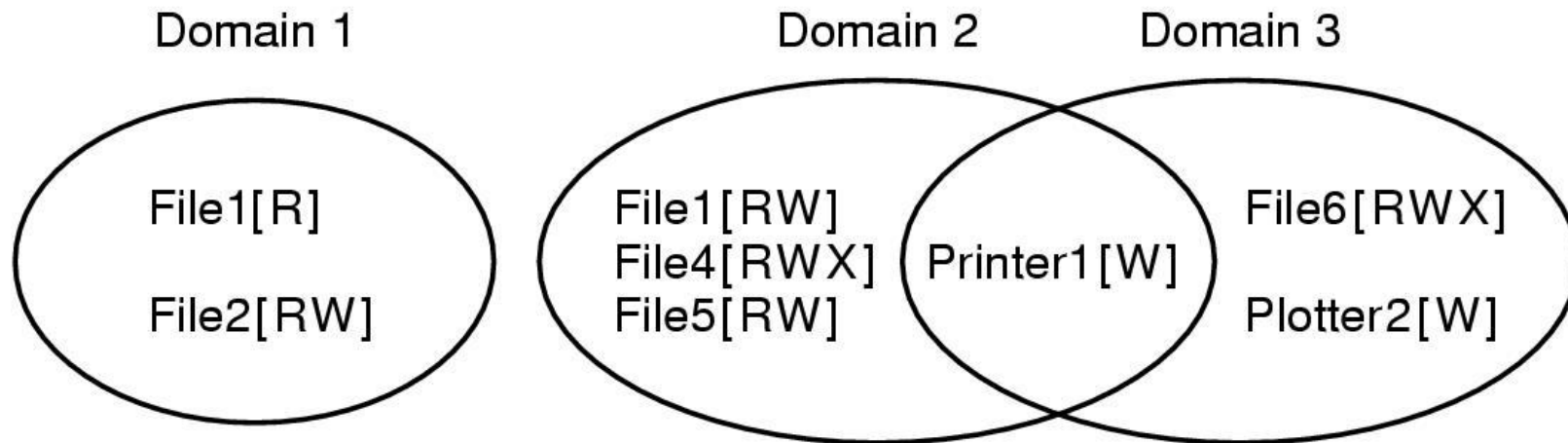
(a) Computing a signature block.

(b) What the receiver gets.

Hashed Passwords

- Store $f(PW)$, where f is not invertible n when user enters PW , calculate $f(PW)$ and compare
- To guard against precomputation attacks, assign a random *salt* at password change time and store $\langle hsalt, f(PW, salt) \rangle$
- Attackers can still run password-guessing programs, so most operating systems use access control to protect the hashed passwords

Protection Domains (1)



Three protection domains

Protection Domains (2)

Domain	Object							
	File1	File2	File3	File4	File5	File6	Printer1	Plotter2
1	Read	Read Write						
2			Read	Read Write Execute	Read Write		Write	
3						Read Write Execute	Write	Write

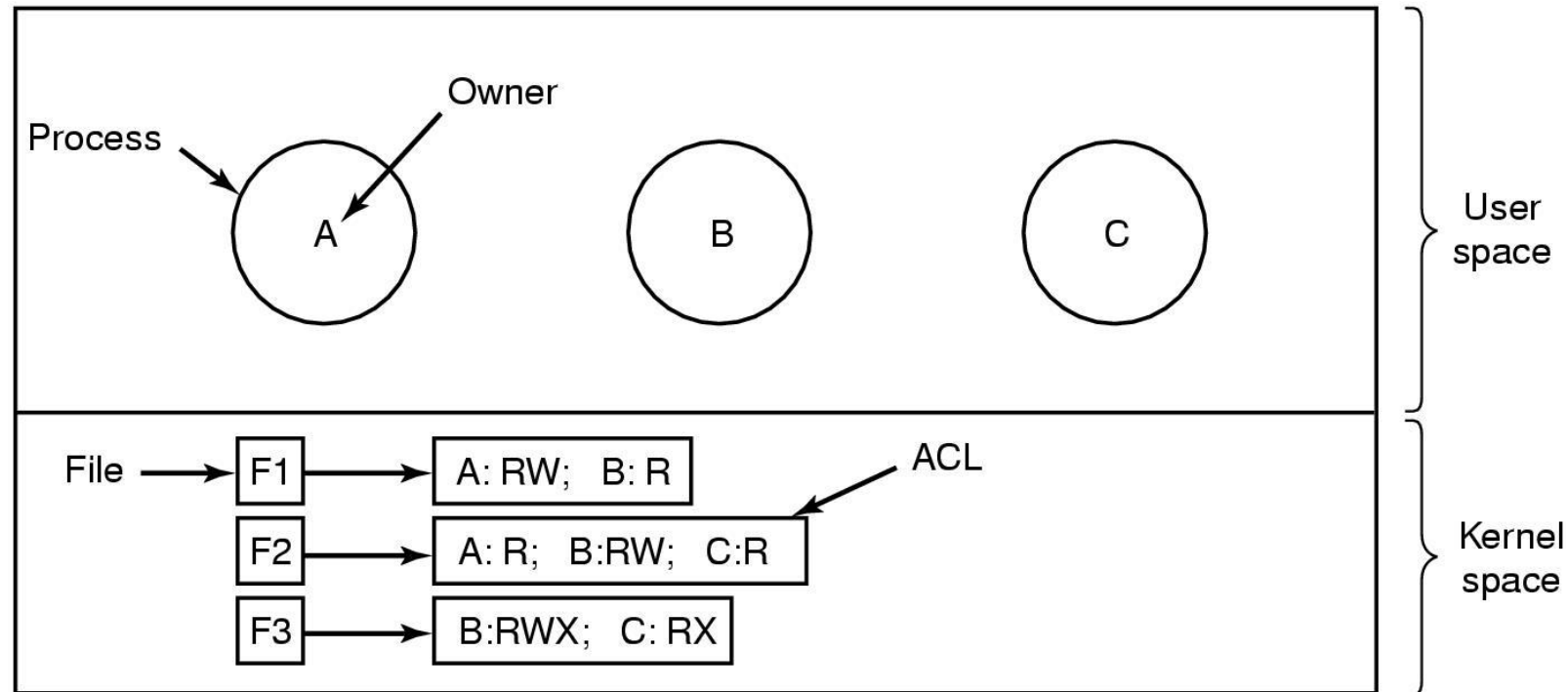
A protection matrix

Protection Domains (3)

	Object										
	File1	File2	File3	File4	File5	File6	Printer1	Plotter2	Domain1	Domain2	Domain3
Domain 1	Read	Read Write								Enter	
2			Read	Read Write Execute	Read Write		Write				
3						Read Write Execute	Write	Write			

A protection matrix with domains as objects

Access Control Lists (1)



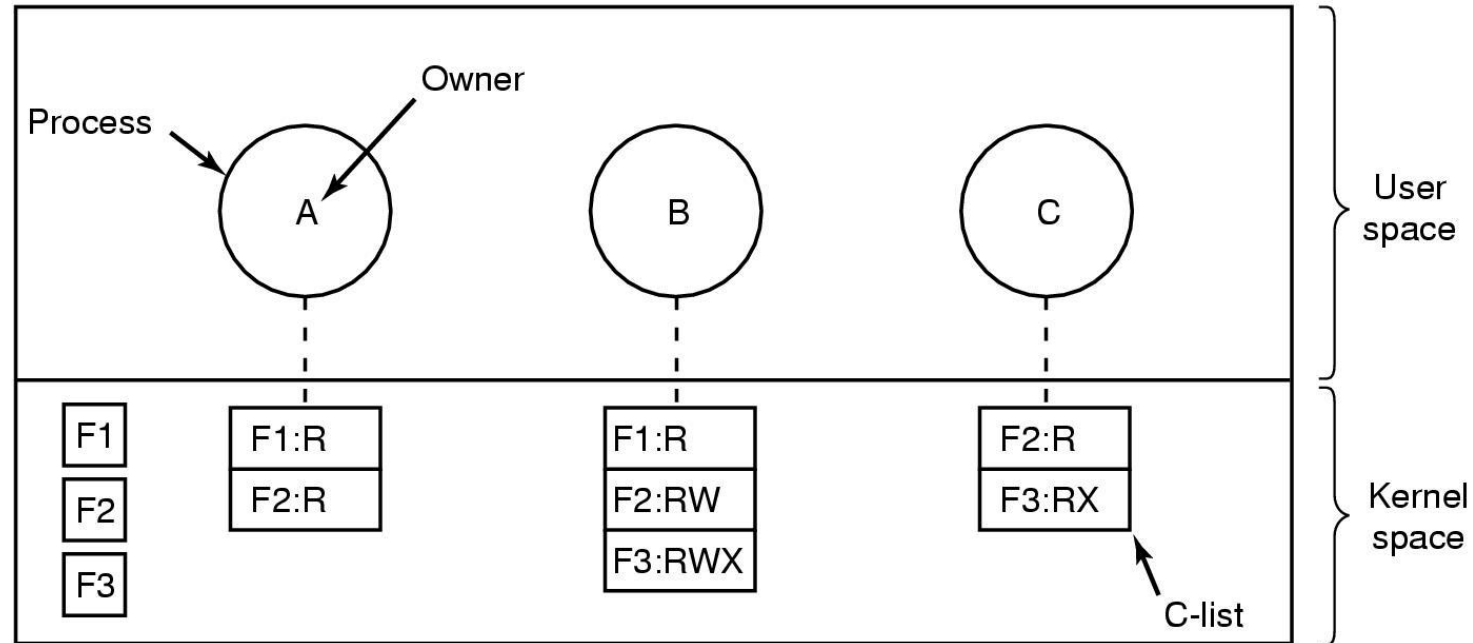
Use of access control lists to manage file access

Access Control Lists (2)

File	Access control list
Password	tana, sysadm: RW
Pigeon_data	bill, pigfan: RW; tana, pigfan: RW; ...

Two access control lists

Capabilities (1)



When capabilities are used, each process has a capability list

Capabilities (2)

Server	Object	Rights	$f(\text{Objects}, \text{Rights}, \text{Check})$
--------	--------	--------	--

A cryptographically protected capability

Capabilities (3)

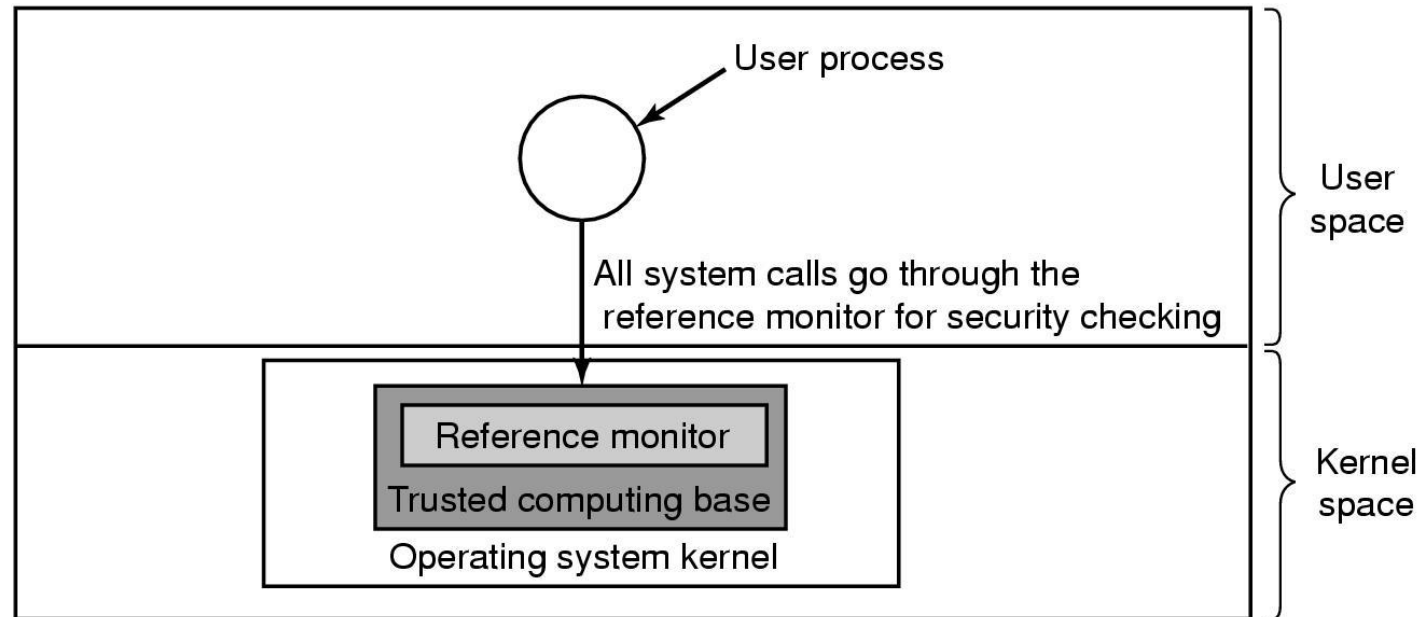
Examples of generic rights:

- Copy capability: create a new capability for the same object.
- Copy object: create a duplicate object with a new capability.
- Remove capability: delete an entry from the C-list; object unaffected.
- Destroy object: permanently remove an object and a capability.

Trusted Systems

- Consider reports of viruses, worms, etc.
- Two naive (but logical) questions:
 - Is it possible to build a secure computer system?
 - If so, why is it not done?

Trusted Computing Base



A reference monitor

Formal Models of Secure Systems

Objects			
	Compiler	Mailbox 7	Secret
Eric	Read Execute		
Henry	Read Execute	Read Write	
Robert	Read Execute		Read Write

(a)

Objects			
	Compiler	Mailbox 7	Secret
Eric	Read Execute		
Henry	Read Execute	Read Write	
Robert	Read Execute	Read	Read Write

(b)

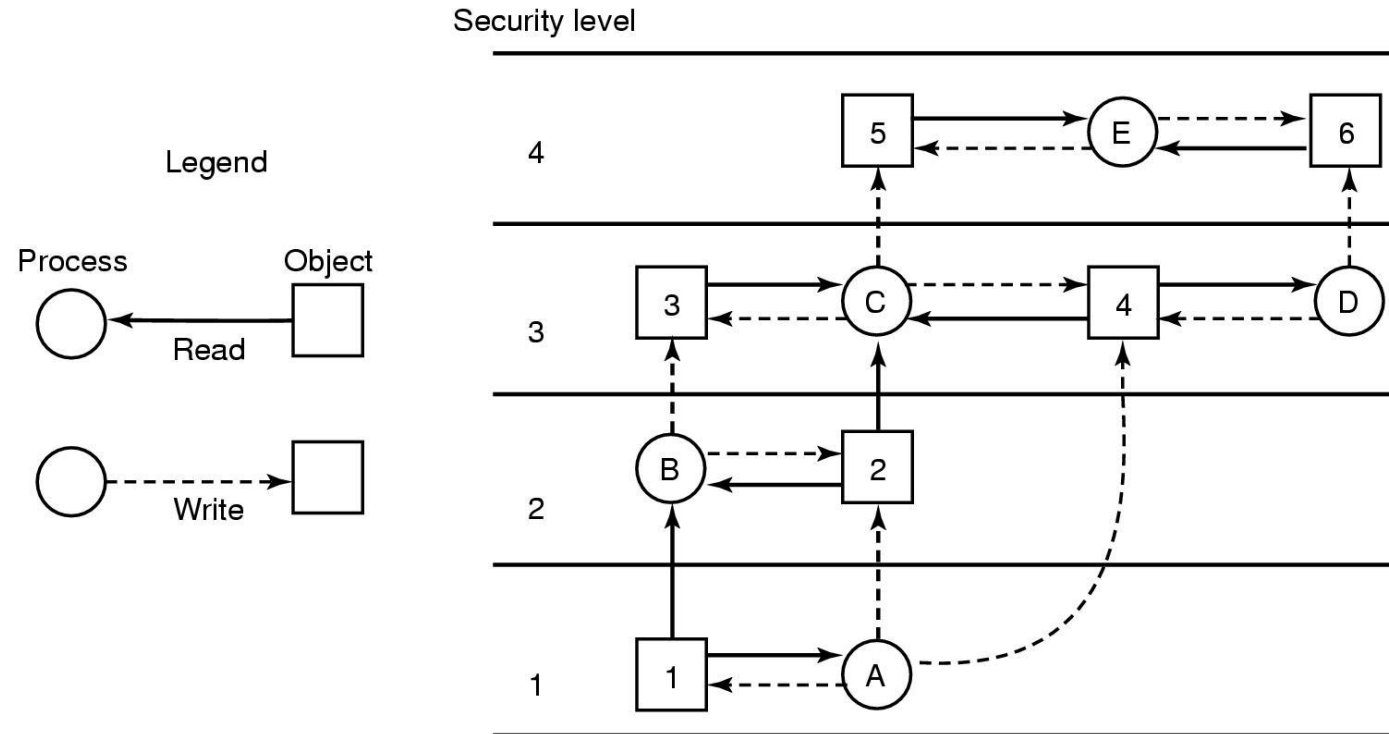
- (a) An authorized state
- (b) An unauthorized state

The Bell-La Padula Model (1)

Rules for the Bell-La Padula model:

- The simple security property: A process running at security level k can read only objects at its level or lower.
 - E.g., a general can read a major's document but not vice versa
- The * property: A process running at security level k can write only objects at its level or higher.
 - E.g., a major can append a message to general's mail box telling everything he knows. The general doing the same may compromise what he knows but major should not..

The Bell-La Padula Model (2)



The Bell-La Padula multilevel security model

The Bell-La Padula Model (3)

- Designed to keep secrets
- How about data integrity?

The Biba Model

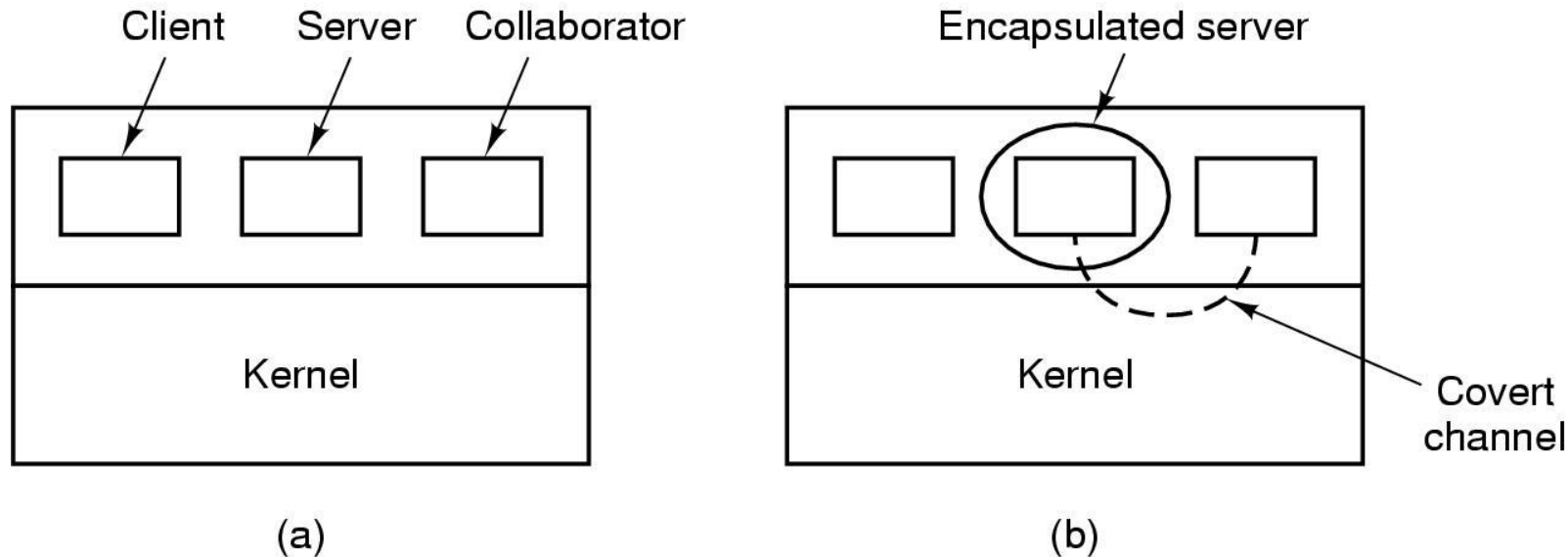
Rules for the Biba model:

- The simple integrity principle: A process running at security level k can write only objects at its level or lower (no write up).
- The integrity $*$ property: A process running at security level k can read only objects at its level or higher (no read down).

How about both?

- Want both Bell-La Padula and Biba properties...

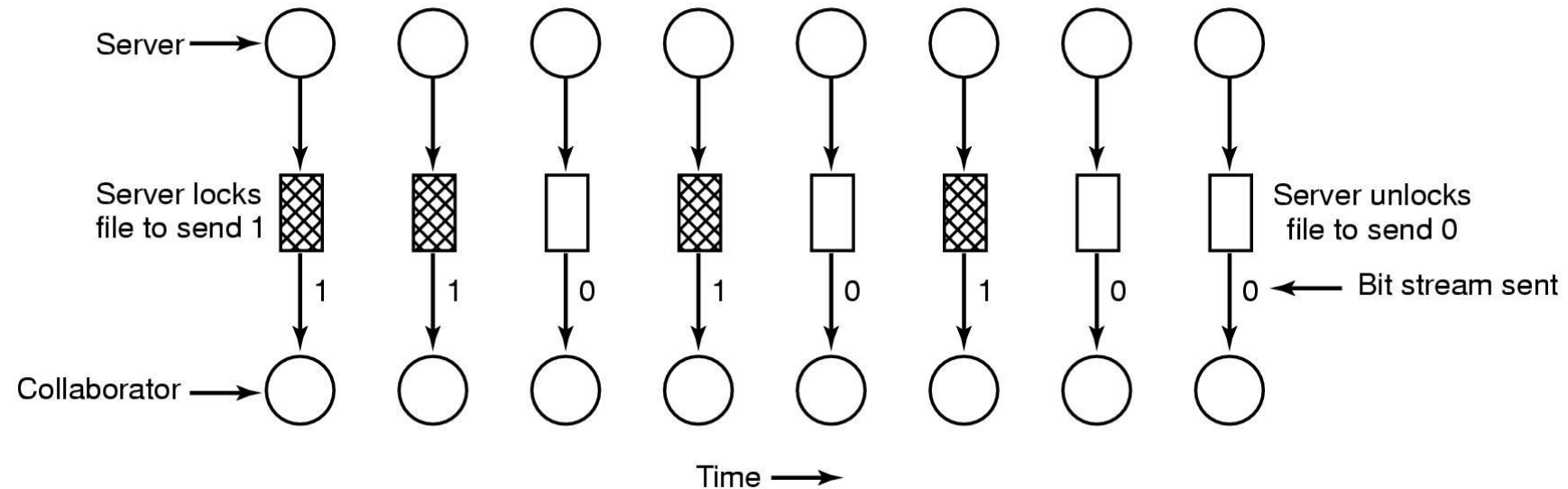
Covert Channels (1)



Confinement Problem

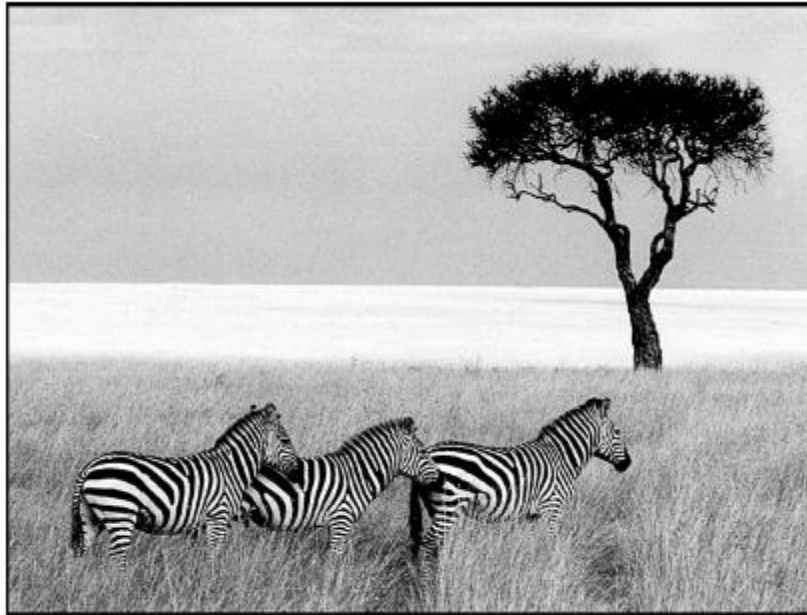
- (a) The client, server, and collaborator processes.
- (b) The encapsulated server can still leak to the collaborator via covert channels.

Covert Channels (2)



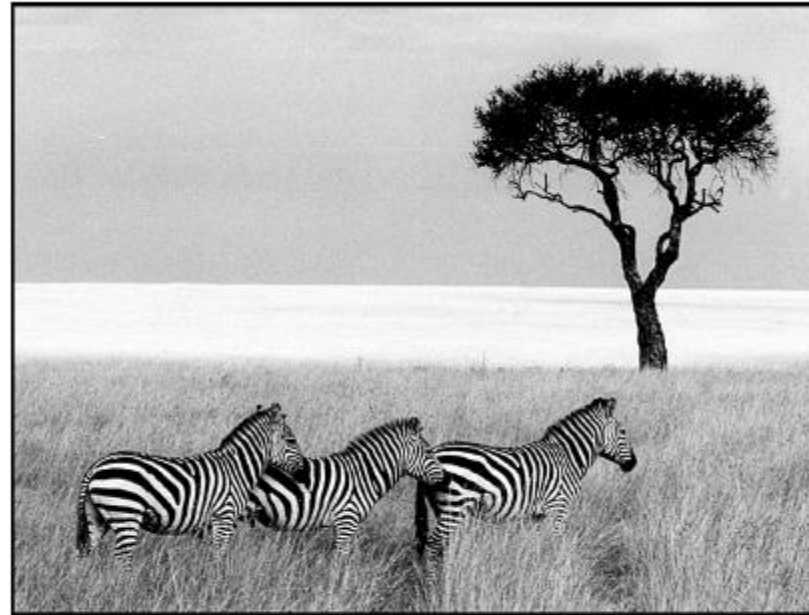
A covert channel using file locking.

Covert Channels (3)



(a)

(a) Three zebras and a tree.



(b)

(b) Three zebras, a tree, and the complete text of five plays by William Shakespeare.

Authentication

General principles of authenticating users:

- Something the user knows.
- Something the user has.
- Something the user is.

Authentication Using Passwords

LOGIN: mitch
PASSWORD: FooBar!-7
SUCCESSFUL LOGIN

(a)

LOGIN: carol
INVALID LOGIN NAME
LOGIN:

(b)

LOGIN: carol
PASSWORD: Idunno
INVALID LOGIN
LOGIN:

(c)

(a) A successful login.

(b) Login rejected after name is entered.

(c) Login rejected after name and password are typed.

How Crackers Break In

```
LBL> telnet elxsi
ELXSI AT LBL
LOGIN: root
PASSWORD: root
INCORRECT PASSWORD, TRY AGAIN
LOGIN: guest
PASSWORD: guest
INCORRECT PASSWORD, TRY AGAIN
LOGIN: uucp
PASSWORD: uucp
WELCOME TO THE ELXSI COMPUTER AT LBL
```

How a cracker broke into a U.S. Department of Energy computer at LBL.

UNIX Password Security

Bobbie, 4238, e(Dog, 4238)
Tony, 2918, e(6%%TaeFF, 2918)
Laura, 6902, e(Shakespeare, 6902)
Mark, 1694, e(XaB#Bwcz, 1694)
Deborah, 1092, e(LordByron,1092)

The use of salt to defeat precomputation of encrypted passwords.

Challenge/Response Authentication

- Ask the user questions no one else would know the answer to
- Note: your mother's maiden name doesn't qualify!
- If the user has calculating ability, do it cryptographically
- The server knows PW and sends a random number N
- Both sides calculate $f(PW, N)$, where f is something like an encryption algorithm
- Note that an eavesdropper who sees N and $f(PW, N)$ can still do password-guessing

Challenge-Response Authentication

The questions should be chosen so that the user does not need to write them down.

Examples:

- Who is Marjolein's sister?
- On what street was your elementary school?
- What did Mrs. Woroboff teach?

The Human Element

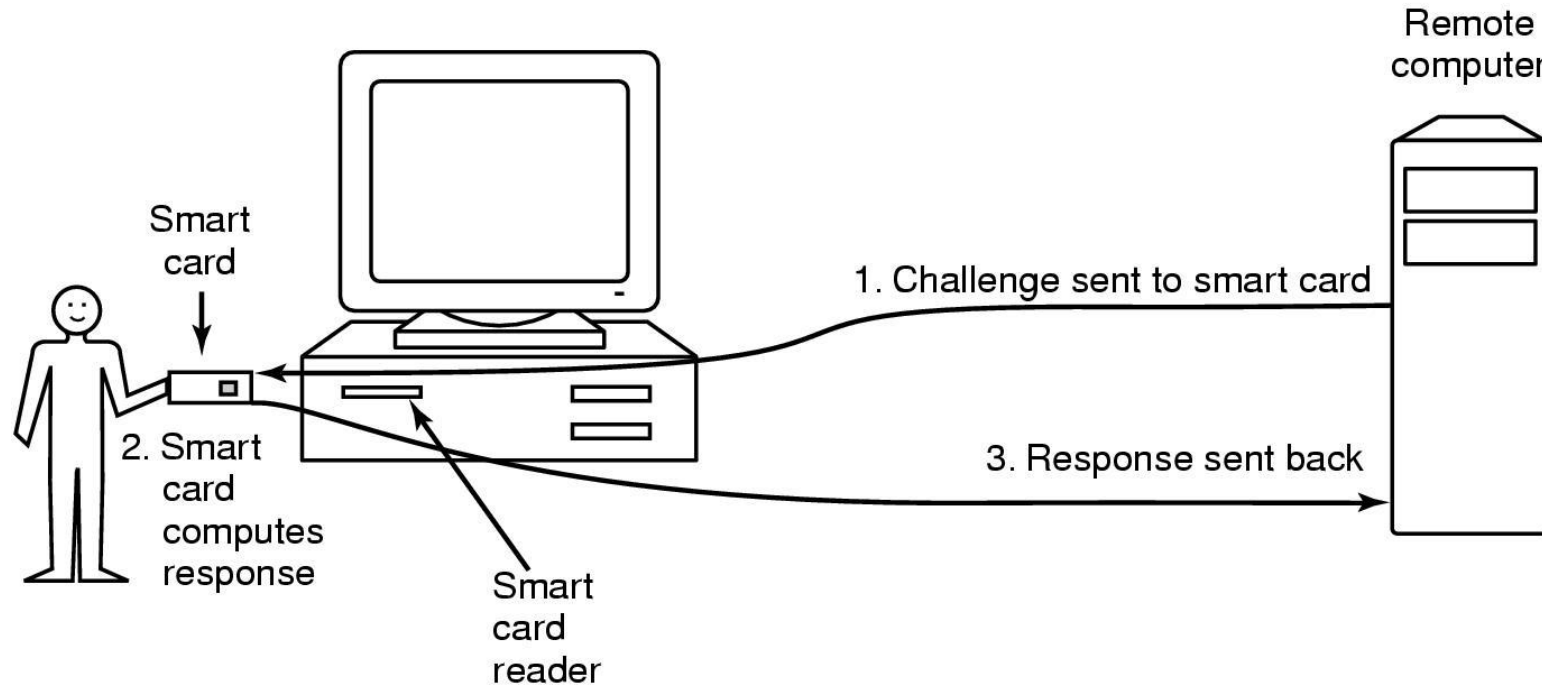
“Humans are incapable of securely storing high-quality cryptographic keys, and they have unacceptable speed and accuracy when performing cryptographic operations. They are also large, expensive to maintain, difficult to manage, and they pollute the environment. It is astonishing that these devices continue to be manufactured and deployed, but they are sufficiently pervasive that we must design our protocols around their limitations.”

—Network Security: Private Communication in a
Public World

Something You Have: Tokens

- Some sort of (generally tamper-resistant) device
- Perhaps it does the challenge/response
- A very popular one has a cryptographic secret and the time of day, and calculates $f(K, T)$
- Often combined with a PIN, to guard against loss or theft

Authentication Using a Physical Object

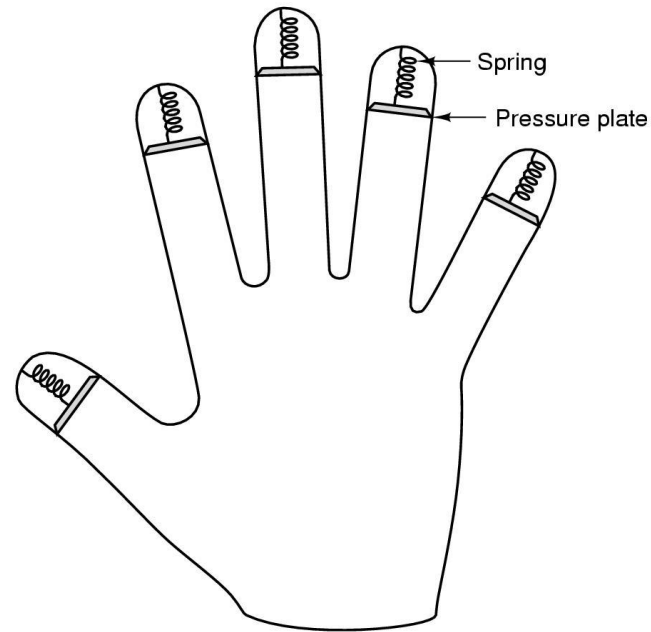


Use of a smart card for authentication

Something You Are: Biometrics

- Fingerprint readers are becoming common
- Iris scans are probably more secure
- Biometrics generally have a false positive rate and a false negative rate — be careful how you set your parameters!
- Biometrics works best if stored locally — over the network, all that's seen is a string of bits
- Watch out for spoofing attacks

Authentication Using Biometrics



A device for measuring finger length

Attack Techniques

- Trojan horses — “come and get it” attack
- Login spoofing
- Buggy software — the big one

Trojan Horses

- Trick someone into executing a program that does nasty things
- (Many viruses and worms spread that way)
- How can the OS protect users?
- Unix-type file permissions don't help — the attack program can change permissions
- Need mandatory access control (MAC)

Trap Doors

```
while (TRUE) {  
    printf("login: ");  
    get_string(name);  
    disable_echoing( );  
    printf("password: ");  
    get_string(password);  
    enable_echoing( );  
    v = check_validity(name, password);  
    if (v) break;  
}  
execute_shell(name);
```

(a)

```
while (TRUE) {  
    printf("login: ");  
    get_string(name);  
    disable_echoing( );  
    printf("password: ");  
    get_string(password);  
    enable_echoing( );  
    v = check_validity(name, password);  
    if (v || strcmp(name, "zzzzz") == 0) break;  
}  
execute_shell(name);
```

(b)

(a) Normal code. (b) Code with a trap door inserted.

Sandboxes

- A better idea is for the OS to provide sandboxes — an environment where the program can execute but can't affect the rest of the machine
- Strong isolation is conceptually pretty easy — run the program on a separate machine, or under VMware or the like
- There are other, more elegant mechanisms that attempt to provide the same feature at lower cost; most are limited to root
- The trick — and it's a very difficult one — is permitting limited interaction with the outside world while still protecting security

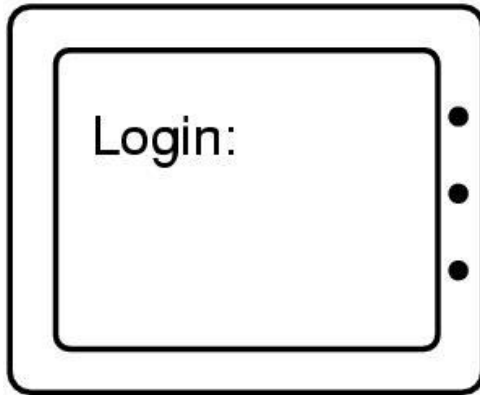
Login Spoofing

NetBSD/i386 (clic) (tty00)

login:

- Is that the real login prompt?
- A fake one could capture your login and password
- (We see similar things today with fake ATMs!)

Login Spoofing



(a)



(b)

(a) Correct login screen. (b) Phony login screen.

Trusted Path

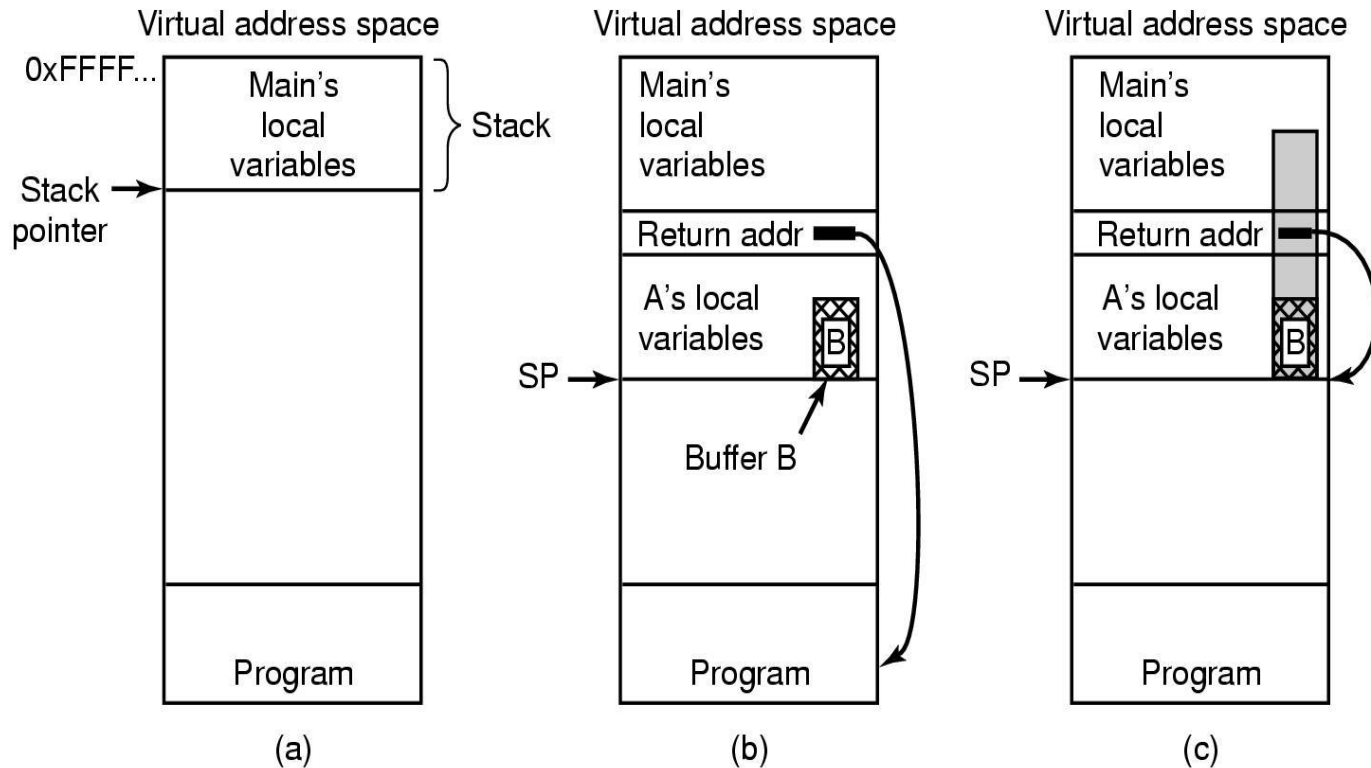
- A trusted path is a user-initiated sequence that is guaranteed to get you to the real OS
- Example: `ctrl+alt+delete` on Windows
- Well, it was supposed to be one. . .
- But — you have to train people not to log in unless they've initiated the sequence
- Must protect all password prompts that way

Exploiting Code Bugs

Example steps to exploit a bug:

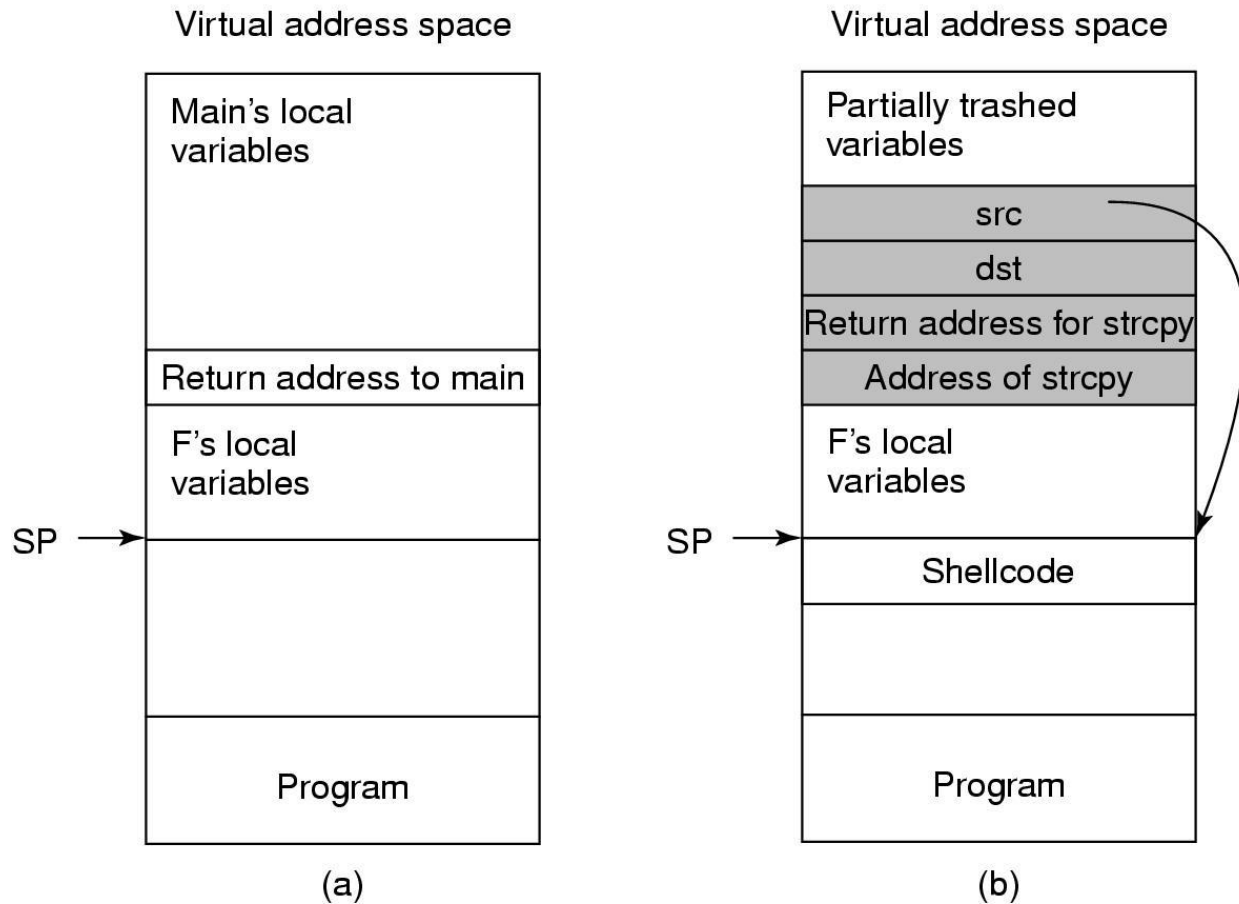
- Run port scan to find machines that accept telnet connections.
- Try to log in by guessing login name and password combinations.
- Once in, run the flawed program with input that triggers the bug.
- If the buggy program is SETUID root, create a SETUID root shell.
- Fetch and start a zombie program that listens to an IP port for cmds.
- Arrange that the zombie program is started when the system reboots.

Buffer Overflow Attacks



- (a) Situation when the main program is running.
- (b) After the procedure A has been called.
- (c) Buffer overflow shown in gray.

Return to libc Attacks



- (a) The stack before the attack.
- (b) The stack after the stack has been overwritten.

Code Injection Attacks

```
int main(int argc, char *argv[])
{
    char src[100], dst[100], cmd[205] = "cp ";    /* declare 3 strings */
    printf("Please enter name of source file: "); /* ask for source file */
    gets(src);                                    /* get input from the keyboard */
    strcat(cmd, src);                             /* concatenate src after cp */
    strcat(cmd, " ");                             /* add a space to the end of cmd */
    printf("Please enter name of destination file: "); /* ask for output file name */
    gets(dst);                                    /* get input from the keyboard */
    strcat(cmd, dst);                             /* complete the commands string */
    system(cmd);                                  /* execute the cp command */
}
```

Code that might lead to a code injection attack

Heartbleed

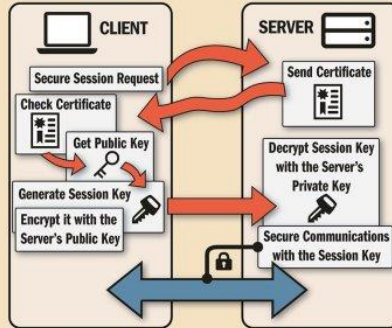


- When you connect with a secure (HTTPS) website, there's a handshake to set up the secure session. Your browser requests and verifies the site's certificate, generates an encryption key for the secure session, and encrypts it using the site's public key. The site decrypts it using the corresponding private key, and the session begins.
- The heartbeat payload is a data packet that includes, among other things, a field that defines the payload length. A Heartbleed attack involves lying about the payload length. The malformed heartbeat packet says its length is 64KB, the maximum possible. When the buggy server receives that packet, it responds by copying that quantity of data from memory into the response packet.
- Just what's in that memory? Well, there's no way to tell. The attacker will have to comb through it looking for patterns. But potentially anything at all could be captured, including encryption keys, login credentials, and more. The fix is simple—check to make sure the sender isn't lying about packet length. Too bad they didn't think to do that in the first place.
- Since exploitation of this bug leaves no traces, we can't really tell how much supposedly-secure data has been stolen. According to statics it looks like most websites are demonstrating the required agility in this case.
- BAE Security reports that on April 8 it found 628 of the top 10,000 websites vulnerable. On April 9, yesterday, that number was down to 301. And this morning it had dwindled to 180.

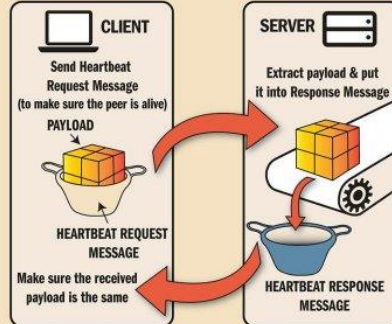
<http://technogرافy.com/2014/04/heartbleed-bug-biggest-security-breach-internet-know/>

HEARTBLEED - THE OPENSSL HEARTBEAT EXPLOIT

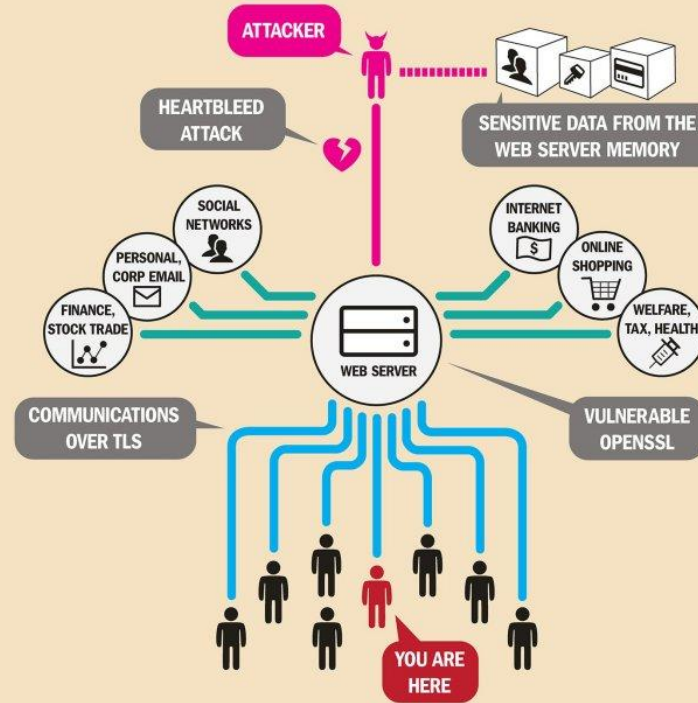
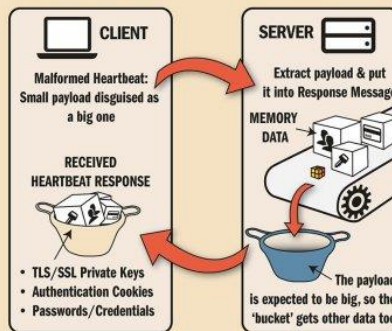
HOW TLS (TRANSPORT LAYER SECURITY) WORKS



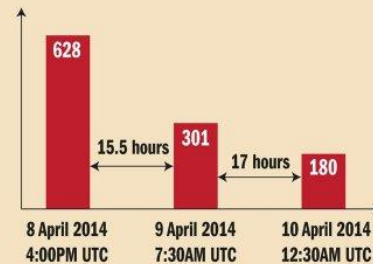
HOW HEARTBEAT EXTENSION FOR TLS WORKS



HOW THE HEARTBLEED EXPLOIT WORKS



NUMBER OF THE VULNERABLE WEBSITES AMONG TOP 10,000



RECOMMENDATIONS

- Check & Upgrade OpenSSL
- Change passwords & keys
- Apply IDS signatures
- Buy a new TLS certificate



Viruses and Worms

- Viruses spread by themselves within a machine, but require human intervention to infect other machines
- Worms spread between machines, though they may require human assistance (i.e., opening an attachment) to infect another machine
- What can the OS do to stop these?

Blocking Executables

- Operating systems can try to block suspicious content
- It's very hard to do — lots of ways to sneak stuff in
- Windows XP SP2 “tags” downloaded files — anything that's tagged is deemed non-executable
- But what about things like bug fixes, that you should permit to be downloaded?

Malware

Can be used for a form of blackmail.

Example: Encrypts files on victim disk, then displays message ...

Greetings from General Encryption

To purchase a decryption key for your hard disk, please send \$100 in small unmarked bills to Box 2154, Panama City, Panama.
Thank you. We appreciate your business.

Types of Viruses

- Companion virus
- Executable program virus
- Parasitic virus
- Memory-resident virus
- Boot sector virus
- Device driver virus
- Macro virus
- Source code virus

Executable Program Viruses (1)

```
#include <sys/types.h>                /* standard POSIX headers */
#include <sys/stat.h>
#include <dirent.h>
#include <fcntl.h>
#include <unistd.h>
struct stat sbuf;                      /* for lstat call to see if file is sym link */

search(char *dir_name)
{
    DIR *dirp;                         /* recursively search for executables */
    struct dirent *dp;                /* pointer to an open directory stream */
                                      /* pointer to a directory entry */

    dirp = opendir(dir_name);          /* open this directory */
    if (dirp == NULL) return;          /* dir could not be opened; forget it */

    ...
```

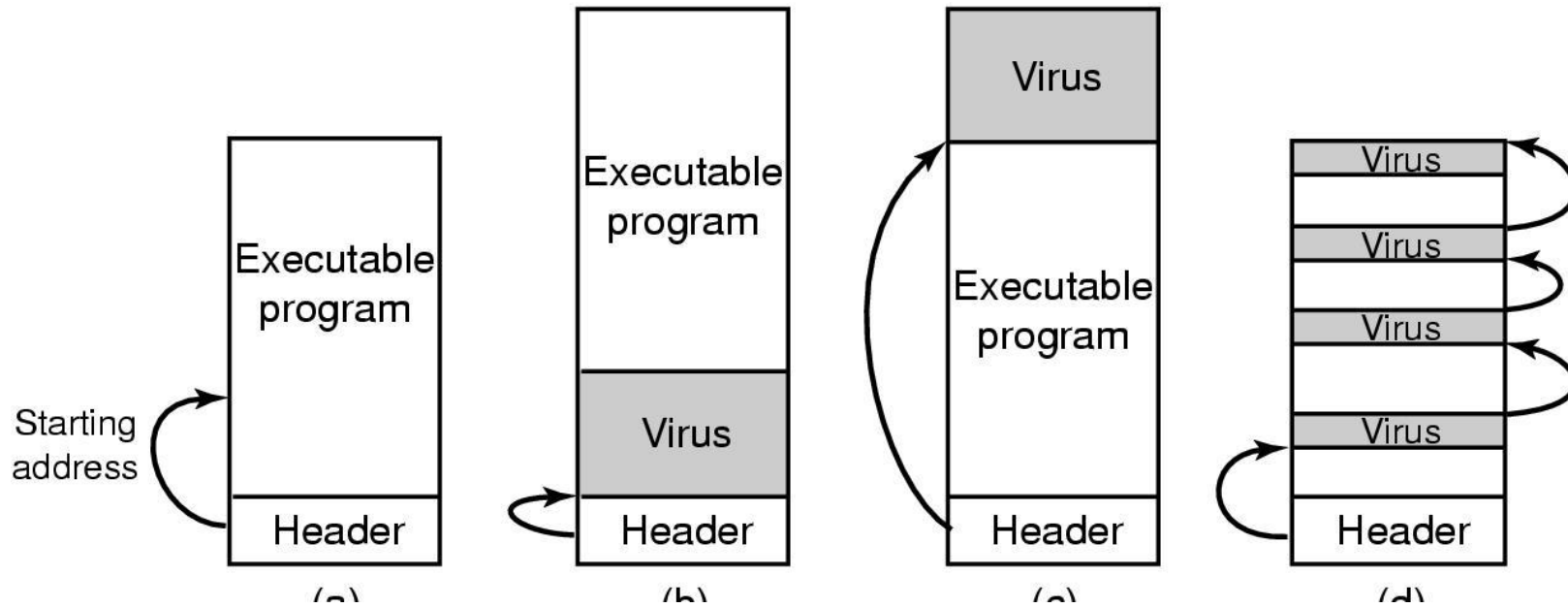
A recursive procedure that finds executable files on a UNIX system

Executable Program Viruses (2)

```
... while (TRUE) {
    dp = readdir(dirp);           /* read next directory entry */
    if (dp == NULL) {             /* NULL means we are done */
        chdir ("..");             /* go back to parent directory */
        break;                   /* exit loop */
    }
    if (dp->d_name[0] == '.') continue; /* skip the . and .. directories */
    lstat(dp->d_name, &sbuf);       /* is entry a symbolic link? */
    if (S_ISLNK(sbuf.st_mode)) continue; /* skip symbolic links */
    if (chdir(dp->d_name) == 0) {   /* if chdir succeeds, it must be a dir */
        search(".");              /* yes, enter and search it */
    } else {                       /* no (file), infect it */
        if (access(dp->d_name, X_OK) == 0) /* if executable, infect it */
            infect(dp->d_name);
    }
    closedir(dirp);               /* dir processed; close and return */
}
```

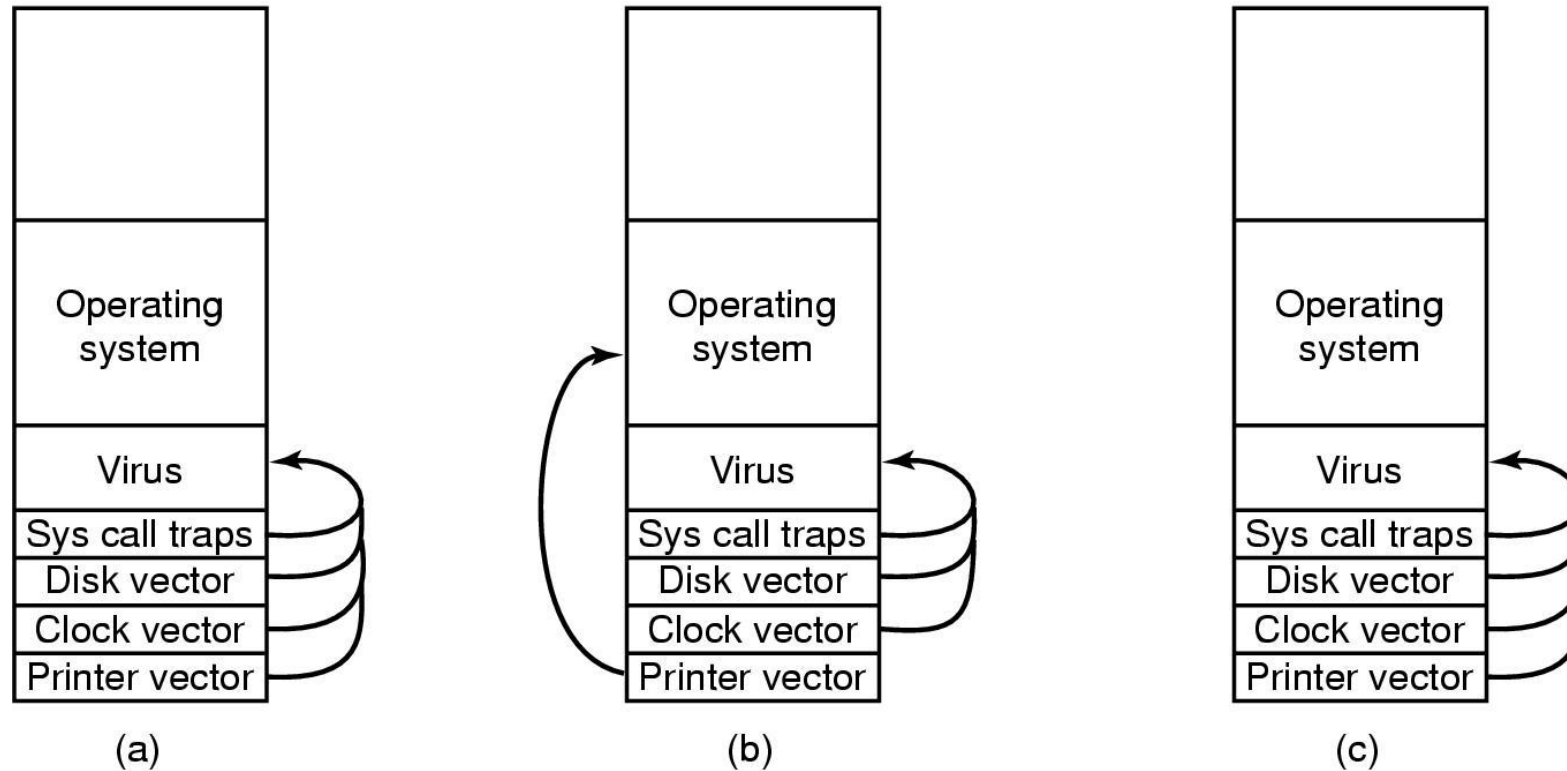
A recursive procedure that finds executable files on a UNIX system

Parasitic Viruses



(a) An executable program. (b) With a virus at the front. (c) With a virus at the end. (d) With a virus spread over free space within the program.

Boot Sector Viruses



(a) After the virus has captured all the interrupt and trap vectors. (b) After the operating system has retaken the printer interrupt vector. (c) After the virus has noticed the loss of the printer interrupt vector and recaptured it.

Spyware (1)

Description:

- Surreptitiously loaded onto a PC without the owner's knowledge
- Runs in the background doing things behind the owner's back

Spyware (2)

Characteristics:

- Hides, victim cannot easily find
- Collects data about the user
- Communicates the collected information back to its distant master
- Tries to survive determined attempts to remove it

How Spyware Spreads

Possible ways:

- Same as malware, Trojan horse
- Drive-by download, visit an infected web site
- Web pages tries to run an .exe file
- Unsuspecting user installs an infected toolbar
- Malicious activeX controls get installed

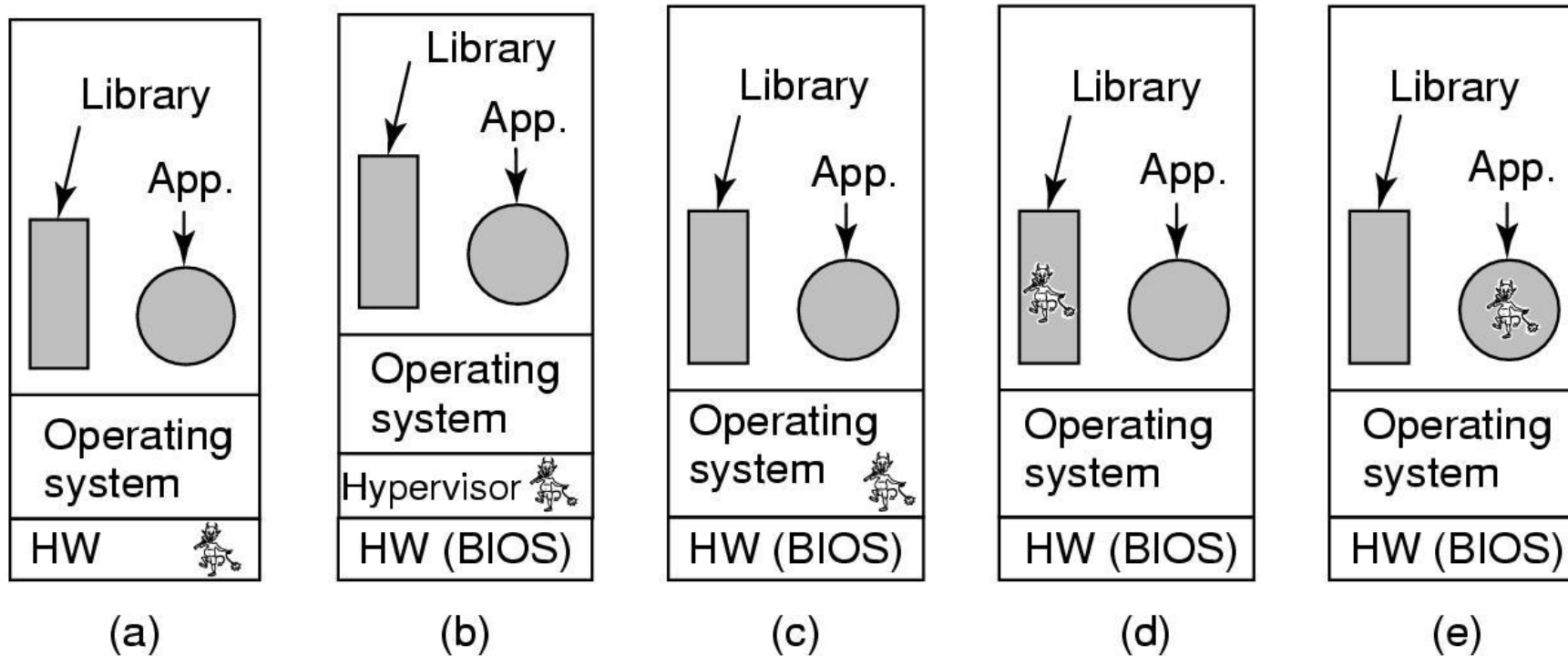
Actions Taken by Spyware

- Change the browser's home page.
- Modify the browser's list of favorite (bookmarked) pages.
- Add new toolbars to the browser.
- Change the user's default media player.
- Change the user's default search engine.
- Add new icons to the Windows desktop.
- Replace banner ads on Web pages with those the spyware picks.
- Put ads in the standard Windows dialog boxes
- Generate a continuous and unstoppable stream of pop-up ads.

Types of Rootkits (1)

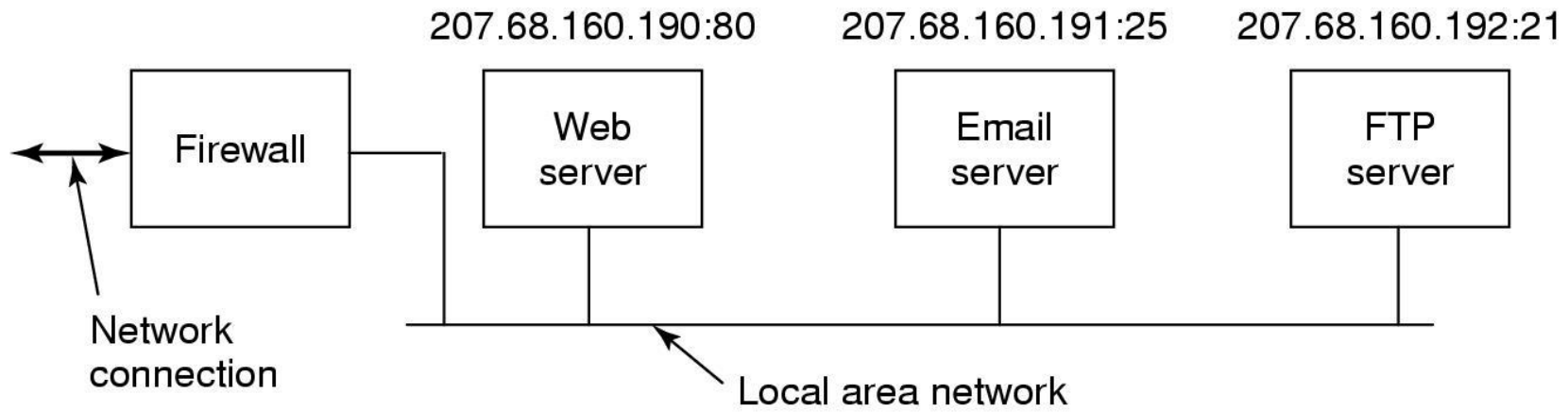
- Firmware rootkits
- Hypervisor rootkits
- Kernel rootkits
- Library rootkits
- Application rootkits

Types of Rootkits (2)



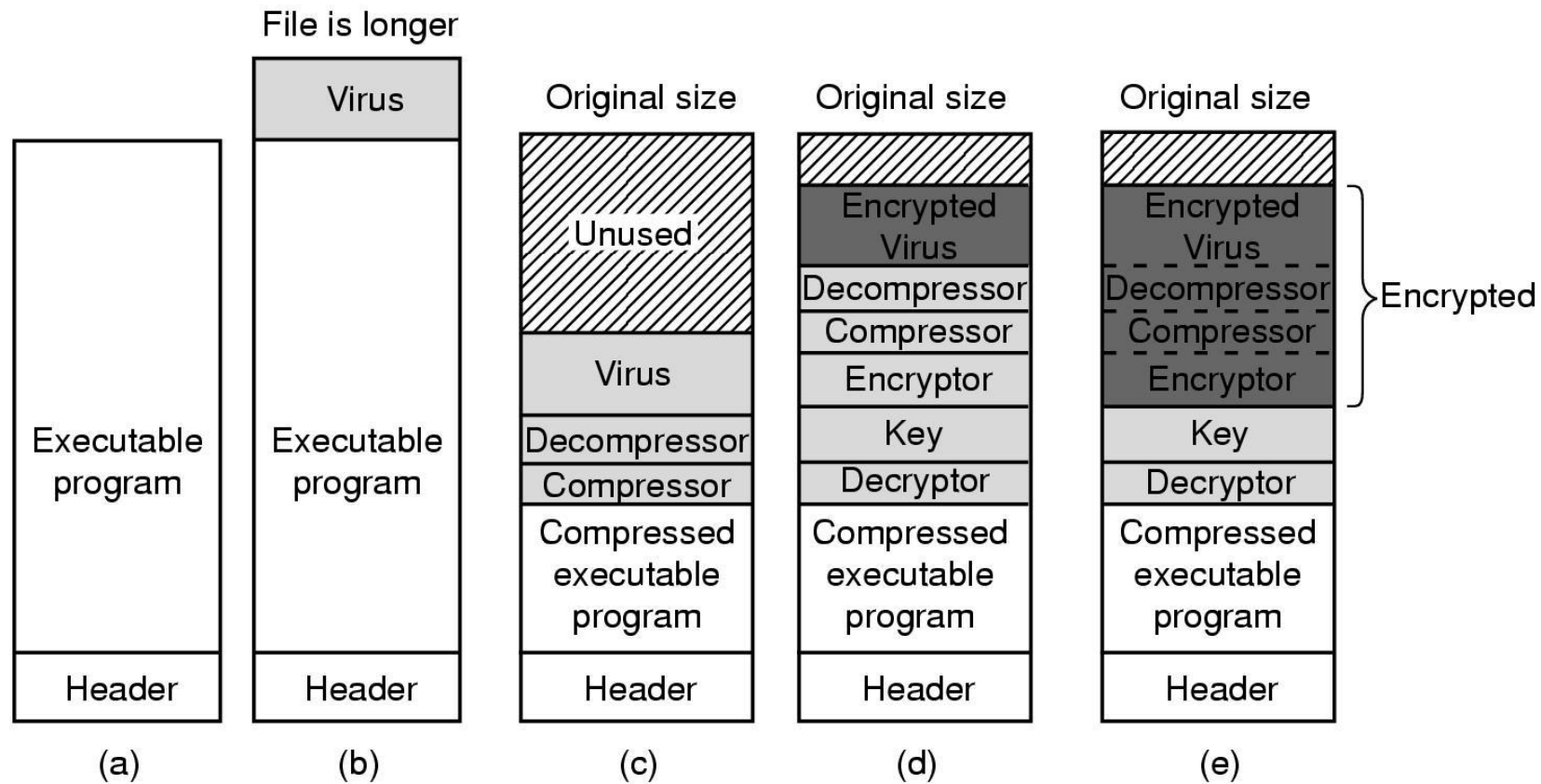
Five places a rootkit can hide.

Firewalls



A simplified view of a hardware firewall protecting a LAN with three computers.

Virus Scanners (1)



a) A program. (b) An infected program. (c) A compressed infected program. (d) An encrypted virus. (e) A compressed virus with encrypted compression code.

Virus Scanners (2)

```
MOV A,R1
ADD B,R1
ADD C,R1
SUB #4,R1
MOV R1,X
```

(a)

```
MOV A,R1
NOP
ADD B,R1
NOP
ADD C,R1
NOP
SUB #4,R1
NOP
MOV R1,X
```

(b)

```
MOV A,R1
ADD #0,R1
ADD B,R1
OR R1,R1
ADD C,R1
SHL #0,R1
SUB #4,R1
JMP .+1
MOV R1,X
```

(c)

```
MOV A,R1
OR R1,R1
ADD B,R1
MOV R1,R5
ADD C,R1
SHL R1,0
SUB #4,R1
ADD R5,R5
MOV R1,X
MOV R5,Y
```

(d)

```
MOV A,R1
TST R1
ADD C,R1
MOV R1,R5
ADD B,R1
CMP R2,R5
SUB #4,R1
JMP .+1
MOV R1,X
MOV R5,Y
```

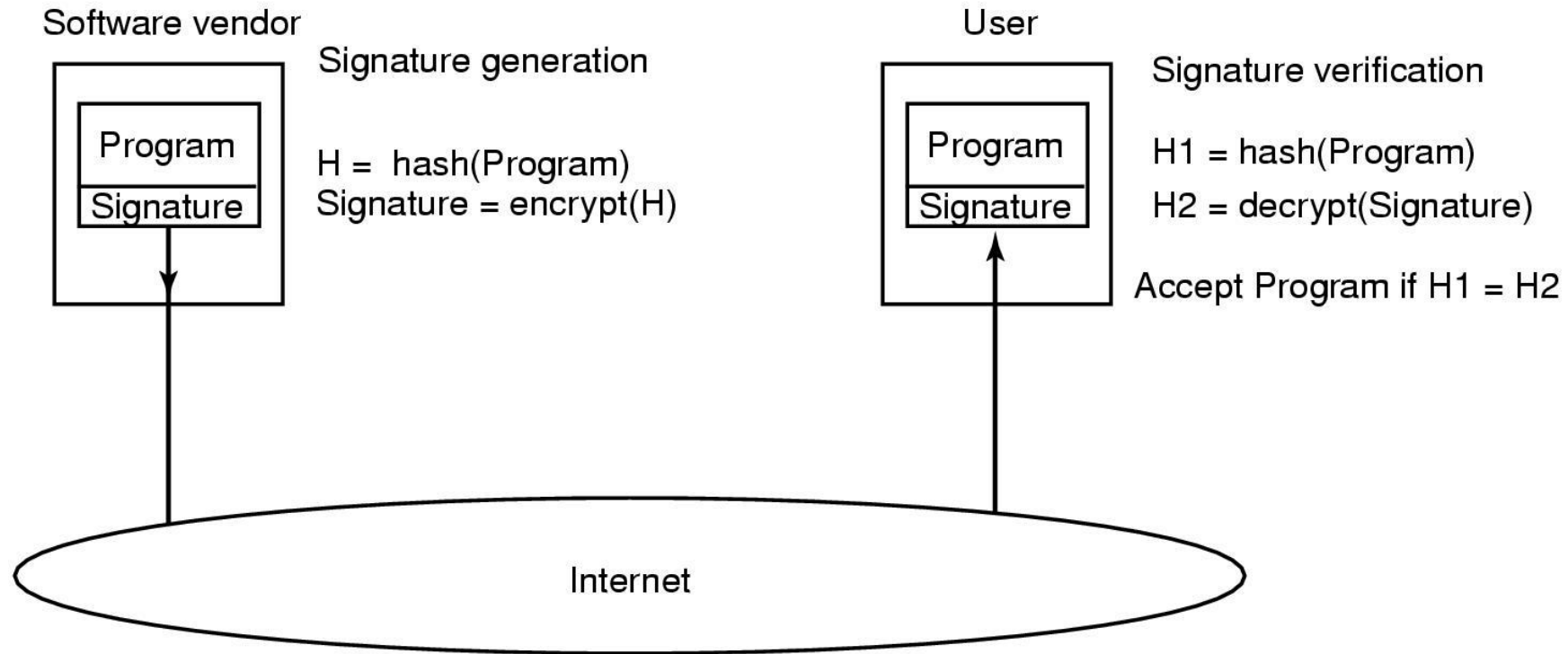
(e)

Examples of a polymorphic virus.

Antivirus and Anti-Antivirus Techniques

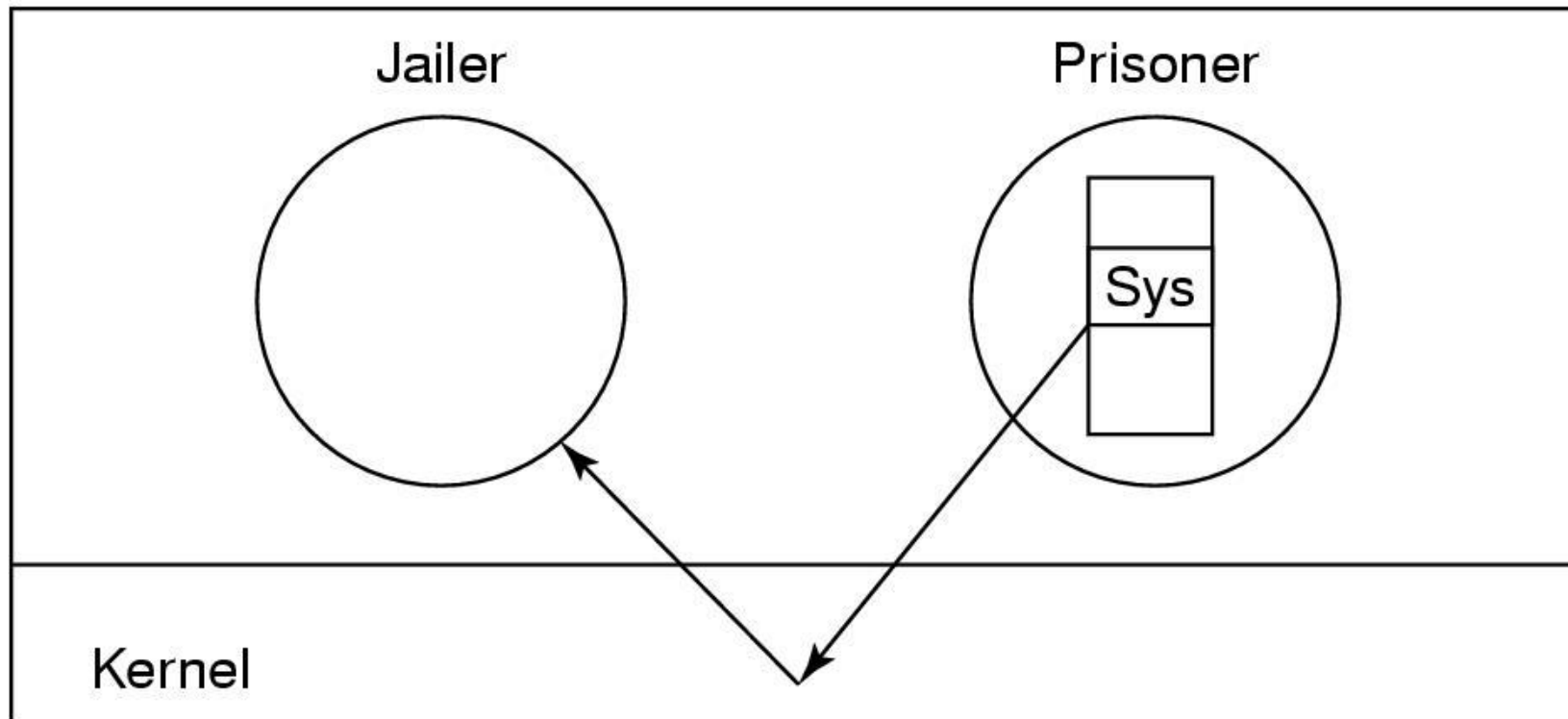
- Virus scanners
- Integrity checkers
- Behavioral checkers
- Virus avoidance

Code Signing



How code signing works.

Jailing



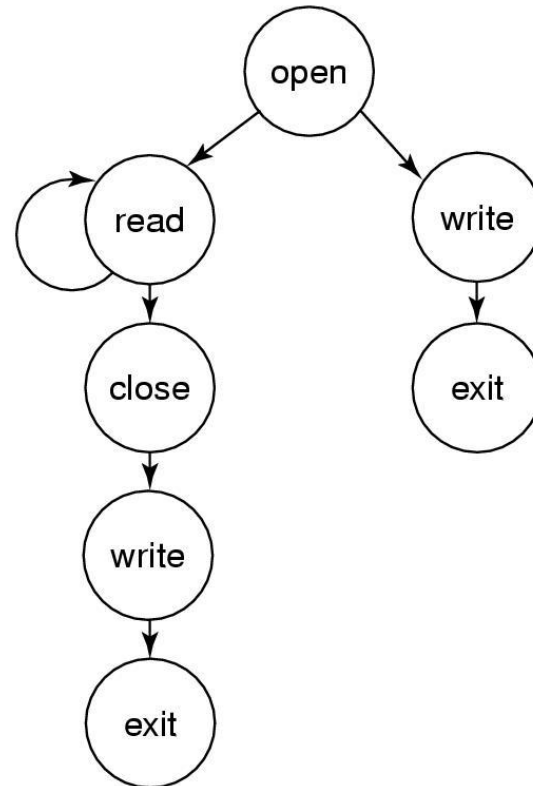
The operation of a jail.

Model-Based Intrusion Detection

```
int main(int argc *char argv[])
{
    int fd, n = 0;
    char buf[1];

    fd = open("data", 0);
    if (fd < 0) {
        printf("Bad data file\n");
        exit(1);
    } else {
        while (1) {
            read(fd, buf, 1);
            if (buf[0] == 0) {
                close(fd);
                printf("n = %d\n", n);
                exit(0);
            }
            n = n + 1;
        }
    }
}
```

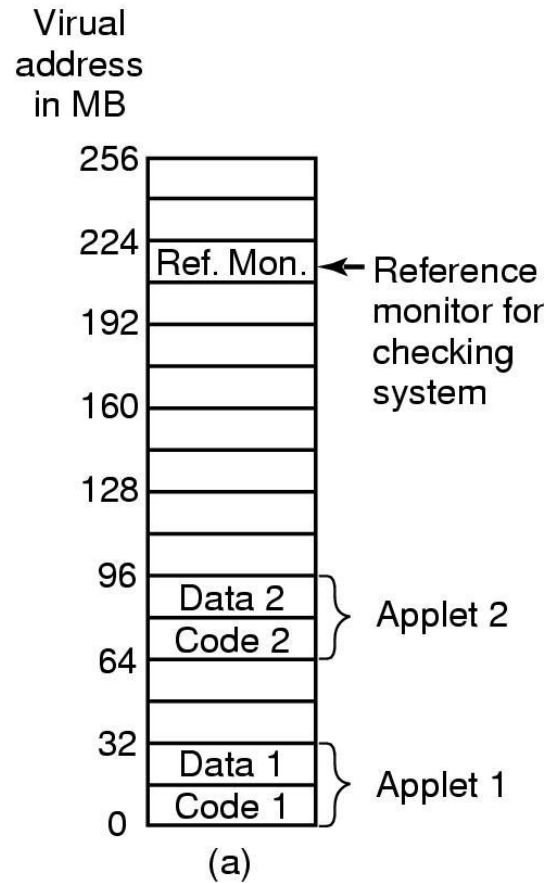
(a)



(b)

(a) A program. (b) System call graph for (a).

Sandboxing



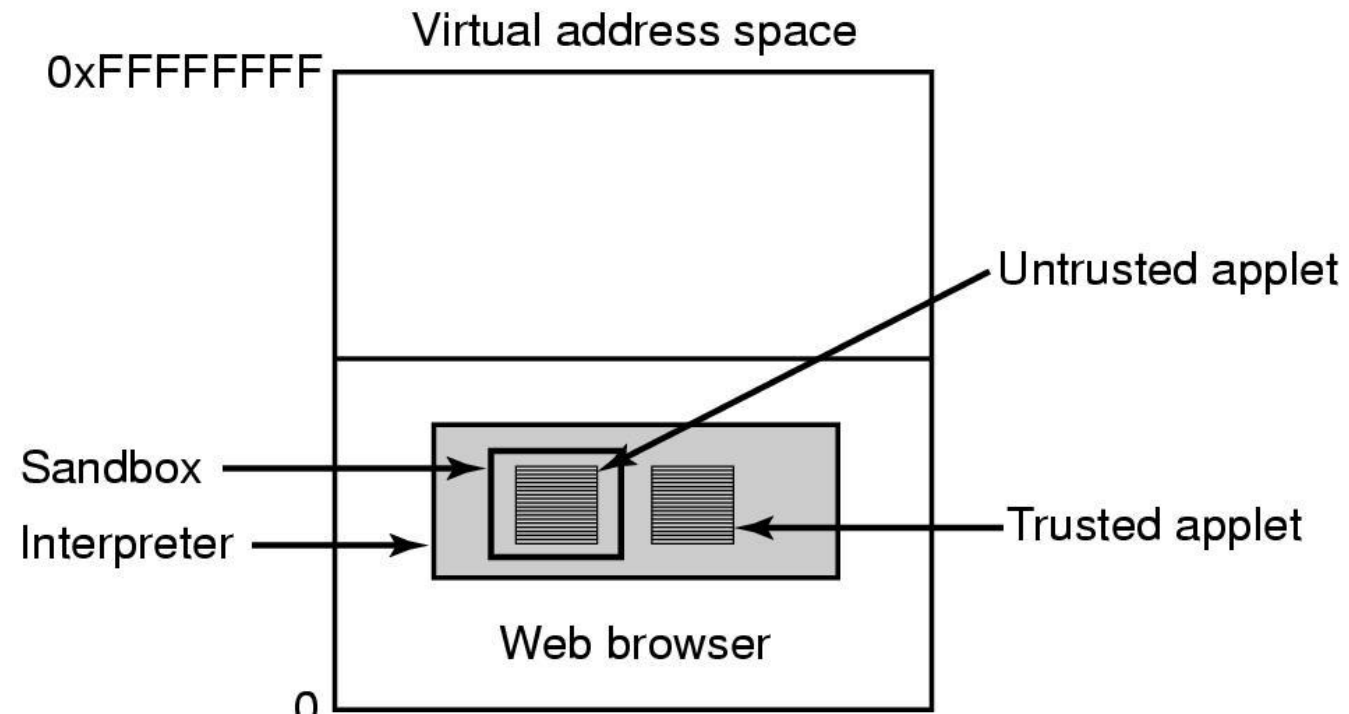
```
MOV R1, S1
SHR #24, S1
CMP S1, S2
TRAPNE
JMP (R1)
```

(b)

(a) Memory divided into 16-MB sandboxes.

(b) One way of checking an instruction for validity.

Interpretation



Applets can be interpreted by a Web browser.

Java Security (1)

JVM byte code verifier checks if the applet obeys certain rules:

- Does the applet attempt to forge pointers?
- Does it violate access restrictions on private-class members?
- Does it try to use a variable of one type as another type?
- Does it generate stack overflows? underflows?
- Does it illegally convert variables of one type to another?

Java Security (2)

URL	Signer	Object	Action
www.taxprep.com	TaxPrep	/usr/susan/1040.xls	Read
*		/usr/tmp/*	Read, Write
www.microsoft.com	Microsoft	/usr/susan/Office/—	Read, Write, Delete

Some examples of protection that can be specified with JDK 1.2.

Access Controls Won't Do It

- One sometimes hears that “Windows is infested with these things because it has no (effective) file protection”
- File protection would prevent OS contamination, but worms can and do spread with user permissions
- The IBM Christmas Card “Virus” (1987) relied on a Trojan Horse emailed shell script
- The Morris Internet Worm (1988) was multi-exploit, multi-platform and didn't violate any OS protections

Certified Systems

- In the early 1980s, the U.S. Defense Department created the so-called Orange Book (DoD Trusted Computer System Evaluation Criteria) and its companions
- The Orange Book described a set of secure system levels, from D (no security) to A1 (formally verified)
- The higher levels had more features; more importantly, they had higher assurance

Military Classification Model

- Documents are classified at a certain level
- People have certain clearances
- You're only allowed to see documents that you're cleared for

Classifications

- Levels: Confidential, Secret, Top Secret
- Compartments: Crypto, Subs, NoForn
- (“NoForn” is “No foreign nationals”)
- To read a document, you must have at least as high a clearance level and you must be cleared for each compartment
- Systems that support this are known as multi-level security systems

Examples

Pat is cleared for Secret, Subs

Chris is cleared for Top Secret, Planes

We have the following files:

warplan	Top Secret	Troops, Subs, Planes
runway	Confidential	Planes
sonar	Top Secret	Subs
torpedo	Secret	Subs

Who can read which file?

Examples

- Pat cannot read warplan; she isn't cleared high enough and she doesn't have Troops or Planes clearance
- Chris can't read it, either; he doesn't have Subs or Planes clearance
- Chris can read runway; Pat can't
- Pat can't read sonar; she has Subs clearance but only at the Secret level
- She can, however, read torpedo

Assurance

- Why do you think the OS is correct?
- How good a job did the developers do?
- Were back doors inserted by nasty developers?
- Higher levels of the Orange Book demanded more assurance, including good documentation, structured design, formal test plans, and security-cleared programmers

The Fate of the Orange Book

- The British, and later Western Europe, produced the Information Technology Security Evaluation Criteria
- The Canadians produced their version
- The US produced the Federal Criteria
- They were all merged into the Common Criteria

The Common Criteria

- The Orange Book mixed features and assurance
- The Common Criteria separated those
- It also permitted varying Protection Profiles
- The Protection Profile says what it's supposed to accomplish
- The feature list says how it does that
- The assurance level is how confident you should be

Bad Models, No Sales

- The Orange Book's security model wasn't interesting commercially
- Getting a system evaluated was time-consuming and expensive
- Evaluated systems lagged well behind commercial ones, and were more expensive
- It was designed for 1980s-style timesharing systems
- “Minor” things like networking weren't included. . .
- Many of the same problems affect Common Criteria systems

Logging

- What's going on on your systems?
- If there's a penetration, will you know?
- Will you be able to figure out how it happened?

Shadow Hawk

Shadow Hawk Busted Again

As many of you know, Shadow Hawk (a/k/a Shadow Hawk 1) had his home searched by agents of the FBI. . .

When he was tagged by the feds, he had been downloading software (in the form of C sources) from various AT&T systems. According to reports, these included the Bell Labs installations at Naperville, Illinois and Murray Hill, New Jersey.

—Phrack Issue 16, File 11, November 1987

How was Shadow Hawk Detected?

- He had broken into some Bell Labs machines
- He tried to use uucp — a dial-up file transfer/email system that came with Unix — to grab /etc/passwd files from other machines
- Uucp logged all file transfer requests
- Several people at Murray Hill had automated jobs that scanned the log files for anything suspicious

What to Log?

- Everything?
- Possibly takes too much storage, though disk space is cheap
- Serious potential privacy risk
- Can you process that much data?
- But — must log security-sensitive events

Solaris Basic Security Module (BSM)

- BSM can do a lot of logging
- Some categories: login/logout, ioctl, file write, network events, mount/unmount, fork, exec, and more
- Great care taken to protect log files
- Again, though — can you process the data?

It's the Application

- The real purpose of an operating system is to run certain applications
- The issue isn't how secure the OS is, it's how secure the applications are
- Again, most worms don't violate OS security

The Challenge

- A useful secure OS should make it easier to write secure applications
- That means things like useful sandboxes
- Need more flexible permission model; DAC is too simple and MAC is too restrictive

We Don't Have Them

- No commercial OS satisfies these criteria
- Of course, most applications are designed for the facilities we do have
- We'll always have buggy code; the trick is to build an application and an OS that will mostly resist attack and will protect the important assets

See you next time!